The Journal of Open Source Software

pyViewFactor: A python library to compute exact view factors between planar faces

Mateusz Bogdan [1*] **and Edouard Walther** [2*]

1 AREP L'hypercube, 16 avenue d'Ivry, 75013 Paris, France **2** ICube Laboratory UMR 7357, Team GCE, Strasbourg University, INSA Strasbourg, CNRS, 67000, Strasbourg,France * These authors contributed equally.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 22 April 2026
Published: unpublished

Summary

pyViewFactor is a Python package for computing view factors — a key component in modeling longwave radiative heat transfer. It supports both full-mesh evaluation and targeted analysis between arbitrary planar polygons. Applications include:

- Improved radiative modeling in Building Energy Simulation,
- Enhanced environmental assessments in urban microclimate analyses,
- Estimation of the mean radiant temperature in comfort studies,
- Implementation of the radiosity method for radiation heat transfer.

The tool is designed for usability, computational efficiency, and adaptability. Given any mesh or set of planar faces, pyViewFactor computes view factors that can then be integrated into simulation workflows, improving the precision of radiant heat transfer modelling.

Statement of Need

The geometric representation of avast majority of shapes in computer aided design (CAD) is made by ensembles of polygons for many fields (architecture and construction, automotive industry, electronics, biomechanics...). pyViewFactor can handle the commonly used input formats such as `.stl`, `.obj` or `.vtk`.

Longwave radiative heat exchange plays a crucial role in thermal simulations across various domains, from building energy performance to urban comfort studies or heat transfer in circuit boards. However, accurate computation of view factors — which uantify the proportion of radiation exchanged between surfaces — remains a challenge due to the absence of analytical solutions for real world geometries and to the expense of the numerical calculation.

Traditional techniques such as Monte Carlo simulations, ray tracing, and hemicube methods are often computationally expensive or limited in resolution. As a result, many simulation tools, such as EnergyPlus or TRNSYS for the building simulation field, rely on simplified assumptions (e.g., isotropic exchange or area-weighted averages) that can overlook critical radiative interactions in urban or interior scenes.

pyViewFactor addresses this gap by offering a fast, accurate, and easy-to-integrate tool for computing view factors between planar surfaces or polygonal meshes. It enables:

- Single facet to facet view factor computation,
- Detailed estimation of Mean Radiant Temperature (MRT) in both indoor and outdoor environments,

- Generation of lookup tables for typical occupant-wall configurations,
- Direct integration into urban-scale simulation or thermal post-processing pipelines, e.g. through the `.vtk` file format.

This tool offers an open-source, computationally efficient alternative that bridges the gap between detailed radiative modeling and practical usability for practicioners and researchers in various fields such as urban climate, building physics, thermal comfort, electronics or more broadly radiation heat transfer.

State of the field

While several tools exist to approximate or compute view factors, most are either embedded in large-scale simulation engines or rely on computationally expensive methods. `pyViewFactor` offers a lightweight and modular alternative that prioritizes numerical accuracy and speed.

Three key related efforts were identified:

- EnergyPlus – supports view factor input but with limited automation (Luo et al., 2020), using the View3D executable developed by (Walton, 2002). The tool works with a spreadsheet as a means of input/output between the executable and the geometric definition of surfaces, which is not especially convenient when dealing with 3D CAD geometries.
- Radiance – provides ray-tracing-based methods, with a focus on daylighting (Ward, 1994). The computation of view factors with Radiance implies a certain level of expertise with the software and the installation of the `radiance` binary which can turn to be challenging on some distributions or operating systems.
- OpenFOAM – the radiation modelling of this computational fluid dynamics software includes and option for the computation of view factor between surfaces. The software is heavy and the view factor calculation does not exist as stand alone tool (Júnior et al., 2018).

Software design

Theory

The core of this package is the use of a quadrature to compute the double contour integral yielding the radiation view factor. The computation method is based on the work of (Mazumder & Ravishankar, 2012) and (Schmid et al., 2019). It was first implemented in Python by (Alecian, 2021) and then published in (Bogdan et al., 2022) with extensive validation and use cases related to urban microclimate. It relies on converting surface integrals (Equation 1) into contour integrals (Equation 2) via *Stokes*' theorem, significantly reducing computational complexity.

$$F_{i,j} = \frac{1}{S_i} \iint_{S_i} \iint_{S_j} \frac{\cos(\theta_i)\cos(\theta_j)}{\delta^2} \, dS_j \, dS_j \tag{1}$$

$$F_{i,j} = \frac{1}{2\pi S_i} \oint_{\Gamma_i} \oint_{\Gamma_j} \ln(\delta_{k,l}) \, d\gamma_l \, d\gamma_k \tag{2}$$

The Journal of Open Source Software

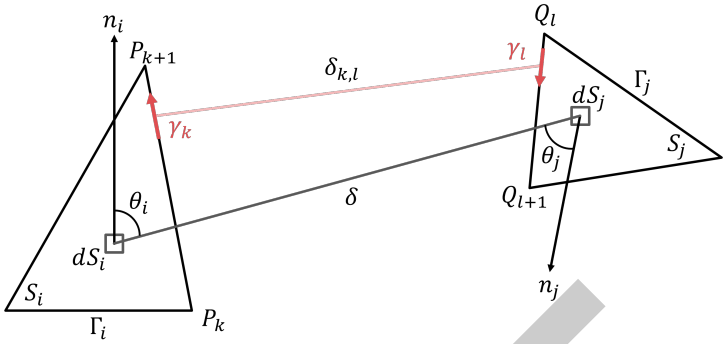

Figure 1: Geometric properties of elemental surfaces

The method applies to planar polygons, consistent with how most digital geometries are discretized into meshes with planar faces. Each polygon boundary is discretised into directed edges, and the double contour integral reduces to a sum over all edge pairs. The inner integral of each pair is evaluated in closed form (SA-30 kernel), so that only the outer integral requires numerical quadrature (30-point Gauss–Legendre rule). This semi-analytic approach handles geometric singularities — adjacent or touching polygons — exactly, without requiring any numerical offset.

Design choices and trade-offs

The design of `pyViewFactor` is driven by a balance between numerical accuracy, computational efficiency, and usability in research workflows.

A key architectural choice is the use of the SA-30 semi-analytic integration kernel for all facet pairs. Unlike a purely numerical approach, the SA-30 kernel evaluates the inner contour integral in closed form for each outer quadrature point, handling both disjoint and touching/coplanar face pairs without any numerical offset or fallback. This eliminates a source of systematic bias present in earlier versions of the library, while maintaining performance through Numba JIT compilation and `prange` parallelism over all pairs.

Obstruction testing — checking whether the centroid-to-centroid ray between two faces is blocked by the obstacle mesh — uses a **Bounding Volume Hierarchy (BVH)** built once at preprocessing time. The BVH organises obstacle triangles into a tree of axis-aligned bounding boxes (AABBs), allowing the traversal to prune entire subtrees with a single slab test. This reduces the per-ray cost from O(N_tri) to O(log N_tri), yielding a combined speedup of approximately 33× over the original implementation on a representative 382-face urban mesh.

Another important design decision is the explicit separation of the pipeline into four independent phases:

- Geometry preprocessing (centroids, normals, areas, BVH construction),
- Visibility filtering (normal-based, vectorised),
- Obstruction filtering (BVH ray traversal),
- View factor integration (SA-30 kernel).

This modular structure allows users to either run the full automated pipeline on a mesh, or manually control each step for debugging, validation, or custom workflows. Visibility and obstruction checks can each be configured in “strict” or “non-strict” modes, allowing users to trade physical rigor against computational cost.

Finally, the reliance on widely adopted scientific Python libraries ensures interoperability with existing research tools and facilitates integration into broader simulation pipelines.

Implementation

pyViewFactor relies on widely used scientific Python libraries to implement the design choices described above:

- Pyvista (Sullivan & Kaszynski, 2019) for 3D geometry handling and visual outputs,
- NumPy (Harris et al., 2020) for vectorized geometrical computations and Gauss–Legendre numerical integration (leggauss),
- SciPy (Virtanen et al., 2020) for the dblquad adaptive integrator, retained for validation and backward compatibility,
- Numba (Lam et al., 2015) for just-in-time compilation and prange parallelism over all face pairs.

Particular attention was given to computational efficiency, combining vectorized operations, closed-form inner integration, and JIT-compiled parallel loops. This results in significant speed-ups compared to naive implementations, while preserving numerical robustness for all geometric configurations.

Research Impact

pyViewFactor has already been used in several independent research contexts, demonstrating its applicability across different domains involving radiative heat transfer:

- Sourisseau et al. (2023): Radiative exchange in greenhouse energy balance modeling,
- Hoffelner et al. (2024): Design of laboratory-scale furnaces for hydrogen plasma smelting,
- Li et al. (2025): Heat loss reduction strategies in the steel industry,
- Žižak et al. (2025): Energy fluxes in PV-integrated green roof analysis,
- Azam et al. (2025): Enhancement and validation of an urban microclimate simulation with the addition of radiation view factors.
- Sourisseau et al. (2026) : Radiative exchange in greenhouse multiphysics dynamic models

These applications cover a wide range of scales, from building components to industrial processes and urban environments, highlighting the versatility of the library.

In addition to these realized applications, pyViewFactor enables reproducible research workflows by providing:

- A fully open-source implementation of contour-integration-based view factor computation,
- Compatibility with standard 3D mesh formats (.stl, .obj, .vtk),
- Integration into existing numerical pipelines via Python.

This combination of demonstrated use cases and reproducible capabilities supports the near-term and ongoing adoption of the library in the research community.

Online documentation

The documentation for this project is available online at:

https://arep-dev.gitlab.io/pyViewFactor/

It is built with MkDocs and mkdocstrings, and includes:

- A theory section covering the view factor integral, the SA-30 semi-analytic kernel, and the BVH obstruction engine,
- An implementation guide covering the pipeline architecture, visibility and obstruction semantics, and performance considerations,
- Step-by-step examples from single face pairs to full urban scene analysis,
- A full API reference auto-generated from numpy-format docstrings.

Getting `pyViewFactor`

`pyViewFactor` is compatible with Python $>=$ 3.10 and supports both Linux and Windows. It can be installed through `pip` as a PyPi package.

AI usage disclosure

Generative AI tools (ChatGPT 4o, OpenAI; Sonnet 4.6, Anthropic) were used to assist in several aspects of the project:

- Completing and improving function docstrings,
- Assisting in drafting and refining unit test cases,
- Providing guidance during the exploration of numerical integration strategies (in particular the use of Gauss–Legendre quadrature, semi-analytic approach and the BVH filtering), which was subsequently independently implemented by the authors,
- Refining wording and improving clarity in the manuscript.

No AI-generated code was used directly without review. All contributions were carefully validated, tested, and verified by the authors to ensure correctness, consistency, and scientific reliability.

Acknowledgements

The authors would like to acknowledge M. Alecian for his initial work on the quadrature code and M. Chapon for her contribution to the code validation.

References

Alecian, M. (2021). *La pluridisciplinarité dans la démarche de conception mise à l'épreuve par l'intromission des expertises : La modélisation thermique de l'environnement urbain au service de l'aménagement ?* [Master's thesis]. ENS Paris Saclay - Ecole d'Urbanisme de la ville de Paris - Ecole des Ponts Paris Tech.

Azam, M.-H., Berger, J., Walther, E., & Guernouti, S. (2025). Design and experimental validation of an urban microclimate tool integrating indoor-outdoor detailed longwave radiative fluxes at district scale. *arXiv Preprint arXiv:2504.01736*. https://doi.org/10.1016/j.enbuild.2025.116370

Bogdan, M., Walther, E., Alecian, M., Chapon, M., & L'hypercube, A. (2022). *Calcul des facteurs de forme entre polygones–application à la thermique urbaine et aux études de confort*. https://ibpsa.fr/jdownloads/Conferences_et_Congres/IBPSA_France/2022_conferenceIBPSA/actes/bogdan-calcul_des_facteurs_de_forme_entre_polygones_-_application___la_thermique_urbaine_et_aux_tudes_de_con.pdf

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk,

M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hoffelner, F., Zarl, M., & Schenk, J. (2024). Development of a new laboratory-scale reduction facility for the hydrogen plasma smelting reduction of iron ore based on a multi-electrode arc furnace concept. *IOP Conference Series: Materials Science and Engineering*, *1309*, 012012. https://doi.org/10.1088/1757-899X/1309/1/012012

Júnior, S. A. V., Scalon, V. L., Avallone, E., & Mioralli, P. C. (2018). Numerical validation of viewFactor and FVDOM radiation models of OpenFOAM® and application in the study of food furnaces. *INGENIARE-Revista Chilena de Ingeniería*, *26*(4). https://doi.org/10.4067/S0718-33052018000400546

Lam, S. K., Pitrou, A., & Seibert, S. (2015). Numba: A llvm-based python jit compiler. *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, 1–6. https://doi.org/10.1145/2833157.2833162

Li, H., Wan, S., Wang, L., Zhao, J., & Ji, D. (2025). Divide and conquer: Spectral-splitting and utilization of thermal radiation from waste heat in the steel industry. *Applied Energy*, *378*, 124836. https://doi.org/10.1016/j.apenergy.2024.124836

Luo, X., Hong, T., & Tang, Y.-H. (2020). Modeling thermal interactions between buildings in an urban context. *Energies*, *13*(9), 2382. https://doi.org/10.3390/en13092382

Mazumder, S., & Ravishankar, M. (2012). General procedure for calculation of diffuse view factors between arbitrary planar polygons. *International Journal of Heat and Mass Transfer*, *55*(23), 7330–7335. https://doi.org/10.1016/j.ijheatmasstransfer.2012.07.066

Schmid, Q., Hachem, E., & Mesri, Y. (2019). A versatile immersed surface-to-surface method for radiation exchange: Implementation and validation. *International Journal of Heat and Mass Transfer*, *134*, 1091–1100. https://doi.org/10.1016/j.ijheatmasstransfer.2019.01.051

Sourisseau, S., Chantoiseau, E. E., Toublanc, C., & Havet, M. (2023). Computing radiative heat transfers in greenhouses: A methodology coupling analytical and numerical approaches for view factors assessment. *Greensys 2023*. https://doi.org/10.17660/ActaHortic.2025.1426.1

Sourisseau, S., Toublanc, C., Chantoiseau, E., & Havet, M. (2026). Contributions to the development and simulations of generic, modular and multiphysics greenhouses dynamic models, evaluated with a whole year study case dataset. *PLOS ONE*, *21*(2), 1–35. https://doi.org/10.1371/journal.pone.0340619

Sullivan, C., & Kaszynski, A. (2019). PyVista: 3D plotting and mesh analysis through a streamlined interface for the visualization toolkit (VTK). *Journal of Open Source Software*, *4*(37), 1450. https://doi.org/10.21105/joss.01450

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., … SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, *17*, 261–272. https://doi.org/10.1038/s41592-019-0686-2

Walton, G. N. (2002). *Calculation of obstructed view factors by adaptive integration*. US Department of Commerce, Technology Administration, National Institute of Standards; Technology. https://doi.org/10.6028/NIST.IR.6925

Ward, G. J. (1994). The RADIANCE lighting simulation and rendering system. *Proceedings of SIGGRAPH*. https://doi.org/10.1145/192161.192286

Žižak, T., Medved, S., & Arkar, C. (2025). Effect of solar photovoltaics on green roof energy balance and evapotranspiration. *Sustainable Cities and Society*, *121*, 106206. https://doi.org/10.1016/j.scs.2025.106206